

sCO4_AT1_DATA INTERPRETATION

1. Real-Time Detection System

A surveillance system uses the following techniques:

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning	95%	Slow

Compare the three techniques and evaluate which method should be selected for a surveillance system operating under strict latency constraints while maintaining acceptable accuracy. Justify your decision considering deployment conditions.

2. Robust Object Detection

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Compare the robustness of the three methods under normal, occluded, and low-light conditions. Evaluate which method provides the best overall performance and discuss the trade-offs involved.

3. Optical Flow Analysis

Two optical flow methods are available:

- Method A: Produces stable motion vectors but misses small motions.
- Method B: Detects fine motion but produces noisy vectors.

Compare the two methods and evaluate which method is more suitable for motion tracking in dynamic scenes. Justify your answer based on accuracy, stability, noise sensitivity, and application requirements.

4. Model Generalization and Overfitting – 10 Marks

Model	Training Accuracy	Testing Accuracy
A	98%	70%
B	90%	88%

Compare the two models and evaluate which model is more reliable for real-world deployment. Explain the generalization ability and overfitting risk of each model and justify your selection.

5. System Performance under Constraints

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate

Model	Accuracy	Latency	Cost
C	95%	120 ms	High

Compare the three models and evaluate which model provides the best balance between accuracy, latency, and cost for a real-time application. Justify your choice considering practical deployment constraints.

ANSWER SCRIPT – DATA INTERPRETATION

CO4 – Assessment Tool 1 | Computer Vision

Question 1: Real-Time Detection System [10 Marks]

Answer structured as: Data Comparison → Evaluation → Justification / Trade-off Discussion.

1. Data Comparison

Method	Accuracy	Speed	Underlying Approach
Viola-Jones	78%	Fast	Haar-like features + cascade of boosted classifiers
YOLO	92%	Moderate	Single-shot CNN, grid-based bounding box regression
Deep Learning (generic, e.g. two-stage CNN)	95%	Slow	Deep feature extraction with region proposals / heavy backbone

Viola-Jones achieves the lowest accuracy (78%) but the highest speed, since it relies on simple, cheap Haar-like features and an early-rejection cascade that discards most non-object windows in the first few stages. YOLO trades a moderate speed cost for a large accuracy jump (92%) by using a single CNN pass to predict all bounding boxes at once. The generic deep learning approach achieves the highest accuracy (95%) but is the slowest, typically because it uses deeper backbones and/or a two-stage region-proposal process that adds significant computation.

2. Evaluation

The requirement is a surveillance system under strict latency constraints while maintaining acceptable accuracy — this is explicitly a constrained optimization, not a pure accuracy maximization. Viola-Jones' speed advantage is real but its 78% accuracy is low enough to risk frequent missed detections and false alarms in a security context, which is a serious operational cost even though it is fast. The 95%-accurate deep learning model, despite being the most accurate, cannot satisfy 'strict latency constraints' since it is explicitly the slowest option — using it would violate the primary constraint stated in the problem.

YOLO occupies the balanced middle ground: it delivers a substantial accuracy improvement over Viola-Jones (92% vs 78%) while remaining fast enough for near-real-time operation (moderate speed, not slow), because its single-pass architecture avoids the heavy multi-stage computation of typical high-accuracy deep detectors.

3. Justification / Decision

YOLO should be selected for this surveillance system. It satisfies the strict latency constraint far better than the generic deep learning model while still providing a large accuracy gain over Viola-Jones, making it the best available compromise between the two competing requirements named in the question. Additional deployment considerations support this choice: modern surveillance hardware (edge GPUs, embedded accelerators) is commonly optimized for single-pass CNN inference of the kind YOLO uses, and a 92% accuracy level is generally sufficient for triggering human review or downstream tracking, whereas the 6–13 percentage-point speed/accuracy gap to the alternatives is a poor trade for a latency-critical application. If hardware were later upgraded or if the surveillance use case shifted toward offline/forensic analysis (where latency no longer matters), the deep learning model's higher accuracy would become the preferable choice instead.

Question 2: Robust Object Detection Under Varying Conditions [10 Marks]

Answer structured as: Data Comparison → Evaluation → Justification / Trade-off Discussion.

1. Data Comparison

Method	Normal	Occlusion	Low Light	Total Drop (Normal → Low Light)
Viola-Jones	80%	60%	55%	25 pts
YOLO	90%	75%	70%	20 pts
Deep Learning	95%	85%	80%	15 pts

All three methods degrade as conditions worsen from normal to occlusion to low light, which is expected since both challenges reduce the amount of usable visual evidence. However, the methods degrade by different amounts: Viola-Jones loses 25 percentage points overall, YOLO loses 20, and deep learning loses only 15 — meaning deep learning is not just the most accurate under ideal conditions, it is also the most robust (smallest relative drop) under adverse ones.

2. Evaluation

Viola-Jones' sharp degradation under occlusion and low light follows directly from its reliance on hand-crafted Haar-like features and a rigid cascade structure — these features assume a relatively clean, well-lit appearance of the object template, so partial occlusion or weak contrast breaks the assumptions the cascade was tuned on, causing many true positives to fail early-stage rejection.

YOLO is more resilient because its learned convolutional features capture more abstract, context-aware patterns than hand-crafted edges, but it still depends on a reasonably intact spatial layout within each grid cell, so occlusion and weak lighting still measurably reduce its confidence and localization accuracy.

Deep learning models (particularly deeper backbones with more representational capacity) tend to learn richer, more invariant feature hierarchies from large and often augmented training data — including examples with partial occlusion or poor lighting — which explains both their higher ceiling accuracy and their comparatively graceful degradation.

3. Justification / Trade-offs

On raw robustness, deep learning provides the best overall performance across all three conditions (95% / 85% / 80%), making it the strongest choice when accuracy and robustness are the priority and computational cost is not the limiting factor — for example, in offline environmental monitoring, forensic review of recorded footage, or well-resourced server-side processing.

The trade-off is the same one seen in Question 1: this robustness comes at the cost of speed (as established in the Q1 data, deep learning is the slowest of the three). YOLO remains the practical choice when the system must operate in real time and cannot tolerate deep learning's latency, accepting a moderate robustness penalty (5–10 percentage points below deep learning in adverse conditions) in exchange for real-time responsiveness. Viola-Jones should generally be avoided for any application where occlusion or poor lighting is common, since its accuracy falls to 55–60% under those conditions — close to unreliable for a security-critical task — and it should be reserved for constrained, well-lit, unoccluded scenarios (e.g., a fixed-angle entry camera in good lighting) where its speed advantage can be exploited safely.

Question 3: Optical Flow Analysis — Method A vs Method B [10 Marks]

Answer structured as: Data Comparison → Evaluation → Justification / Trade-off Discussion.

1. Data Comparison

Property	Method A	Method B
Motion vector stability	Stable / smooth	Noisy
Sensitivity to small motions	Misses small motions	Detects fine motion
Likely underlying approach	Global / regularized (e.g., Horn–Schunck-style, smoothness-constrained)	Local / high-sensitivity (e.g., Lucas–Kanade at fine scale, or unregularized per-pixel estimation)

Method A and Method B sit at opposite ends of a classic optical-flow trade-off between smoothness and sensitivity. A smoothness-regularized method (typical of Method A's behavior) enforces spatial consistency among neighboring motion vectors, which suppresses noise and produces stable output, but that same regularization acts as a low-pass filter on the motion field, washing out small or subtle motions. A method without such regularization (typical of Method B's behavior) responds faithfully to every local intensity change, including genuine fine motion, but with no mechanism to suppress noise, it is equally responsive to illumination flicker, sensor noise, and compression artifacts, producing a noisy vector field.

2. Evaluation

For motion tracking in dynamic scenes, both weaknesses matter, but they matter differently depending on the type of motion being tracked. Missing small motions (Method A's weakness) is a problem when a scene contains fine or subtle movement that is operationally important — e.g., early detection of a slowly approaching object, or fine gesture tracking — since the tracker will simply fail to register these events at all. Noisy vectors (Method B's weakness) are a different kind of problem: the fine motion is detected, but the noise corrupts the trajectory estimate, which can cause a tracker to lose lock, jitter, or accumulate drift over time, especially in a Kalman-filter or trajectory-smoothing-based tracking pipeline that assumes reasonably clean per-frame measurements.

In most dynamic-scene tracking applications, a completely missed detection is a harder failure to recover from than a noisy-but-present one, because a downstream tracker (e.g., a Kalman filter, particle filter, or temporal smoothing stage) is specifically designed to filter noisy measurements over time, but it cannot act on motion that was never reported in the first place.

3. Justification / Recommendation

Method B is generally more suitable for motion tracking in dynamic scenes, provided its noise is subsequently managed by a filtering or smoothing stage (e.g., temporal averaging, a Kalman filter, or RANSAC-based outlier rejection on the vector field) downstream of the raw optical flow computation. This combination — high-sensitivity flow estimation followed by statistical noise suppression — captures both fine and large motions while still achieving usable trajectory stability, whereas Method A's smoothness comes at the unrecoverable cost of blindness to small motion.

Exception: if the application specifically tracks large, coarse-scale motion (e.g., overall crowd flow direction, or vehicle-level tracking where sub-pixel fine motion is irrelevant) and computational resources for post-filtering are unavailable, Method A's inherent stability becomes the more practical, lower-effort choice. The final decision should therefore be guided by whether the application's motions of interest are predominantly fine-scale (favoring Method B + filtering) or coarse-scale (favoring Method A alone).

Question 4: Model Generalization and Overfitting — Model A vs Model B [10 Marks]

Answer structured as: Data Comparison → Evaluation → Justification / Trade-off Discussion.

1. Data Comparison

Model	Training Accuracy	Testing Accuracy	Train–Test Gap
A	98%	70%	28 pts
B	90%	88%	2 pts

Model A achieves near-perfect training accuracy (98%) but its testing accuracy drops sharply to 70%, a 28-point gap. Model B has a lower training accuracy (90%) than Model A, but its testing accuracy (88%) is far closer to its training accuracy, with only a 2-point gap.

2. Evaluation — Generalization and Overfitting Risk

The size of the train–test gap is the standard diagnostic for overfitting: a large gap means the model has learned patterns specific to the training set — including noise, outliers, or incidental correlations — that do not hold on unseen data, rather than learning the underlying, generalizable structure of the problem. Model A's 28-point gap is a strong, textbook overfitting signature: its extremely high training accuracy suggests the model has enough capacity (or was trained long enough / with too little regularization) to memorize the training set rather than generalize from it.

Model B's much smaller 2-point gap indicates good generalization: its performance on unseen test data is nearly identical to its performance on the data it was trained on, which is exactly the property required for reliable real-world deployment, where the model will constantly face inputs it has never seen before. Model B's slightly lower training accuracy is not a weakness here — it is consistent with a model that has appropriately-limited capacity or regularization, learning genuine patterns rather than fitting noise.

3. Justification / Selection

Model B is more reliable for real-world deployment. Testing accuracy — not training accuracy — is the relevant predictor of real-world performance, since deployment data is by definition unseen data, and Model B's 88% testing accuracy substantially exceeds Model A's 70%. Deploying Model A would be risky: its high training accuracy is misleading and does not reflect how it will actually perform once in production, where it is likely to make far more errors than its training-time metrics suggest.

To make Model A viable, corrective measures would be needed before deployment — e.g., stronger regularization (dropout, weight decay), data augmentation, early stopping, cross-validation-based model selection, or simply more diverse training data — specifically aimed at closing the train–test gap. Until such a gap is closed, Model B should be selected as the deployment candidate, since it already demonstrates that its accuracy will hold up under conditions resembling real-world use, and reliability under previously-unseen input is a more important criterion for deployment than raw training-time accuracy.

Question 5: System Performance Under Accuracy–Latency–Cost Constraints [10 Marks]

Answer structured as: Data Comparison → Evaluation → Justification / Trade-off Discussion.

1. Data Comparison

Model	Accuracy	Latency	Cost	Accuracy Gain / Latency Cost (vs. Model A)
A	88%	50 ms	Low	baseline
B	92%	80 ms	Moderate	+4 pts accuracy for +30 ms, +cost tier
C	95%	120 ms	High	+7 pts accuracy for +70 ms, ++cost tier

Moving from Model A to Model B costs an additional 30 ms of latency and one step up in cost, in exchange for a 4-percentage-point accuracy gain. Moving from Model B to Model C costs a further 40 ms and another cost-tier increase for only a 3-point accuracy gain — a worse marginal return than the A→B step. This pattern (diminishing accuracy return for increasing latency/cost) is common as models approach the top of an accuracy curve.

2. Evaluation

Model A offers the best raw efficiency (lowest latency, lowest cost) but the lowest accuracy of the three; whether 88% is 'acceptable' depends entirely on the application's error tolerance. Model C offers the highest accuracy but at more than double Model A's latency and the highest cost tier — for a real-time application, 120 ms may already be too slow depending on the required frame rate or interaction responsiveness (for context, sub-100 ms latency is typically the target for systems perceived as 'real-time' by users, and well under that for safety-critical control loops).

Model B sits at a genuine middle point: it captures more than half of the total available accuracy improvement (4 of the 7 points between A and C) while only incurring about 43% of the total additional latency (30 of 70 ms) and a smaller cost increase than Model C. This makes B's marginal trade noticeably more efficient than C's.

3. Justification / Recommendation

Model B provides the best overall balance of accuracy, latency, and cost for a real-time application. It delivers a meaningful accuracy improvement over Model A (88% → 92%) while remaining within a latency budget still compatible with real-time operation (80 ms), at only a moderate cost increase — avoiding both Model A's comparatively weak accuracy and Model C's high latency and high cost for a diminishing accuracy return.

This recommendation should be adjusted based on the application's specific constraints: if the system has a hard real-time budget below 80 ms (e.g., a control loop or safety-critical trigger), Model A becomes the only feasible option despite its lower accuracy. Conversely, if the application is not truly latency-critical (e.g., batch analysis, periodic quality auditing) and cost is not a limiting factor, Model C's higher accuracy becomes worth its slower response time. The general principle illustrated across all three models is that the best deployment choice is the one that satisfies the hardest constraint (usually latency, in a 'real-time application') at the highest accuracy achievable within that constraint — which is exactly the position Model B occupies here.